



---

# Logic Design

---

## Ch 2 Part B

# Boolean Algebra and Logic Gates

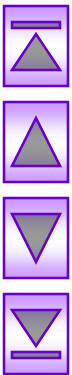
Instructor : Hasan Al Serhan

IT Department

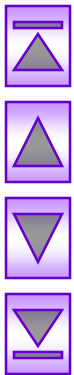
Al Balqa Applied University



- Digital circuits
- Boolean Algebra
- Two-Valued Boolean Algebra
- Boolean Algebra Postulates
- Precedence of Operators
- Truth Table & Proofs
- Duality



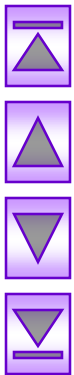
- Basic Theorems of Boolean Algebra
- Boolean Functions
- Complement of Functions
- Standard Forms
- Minterm & Maxterm
- Canonical Forms
- Conversion of Canonical Forms
- Binary Functions



# Introduction

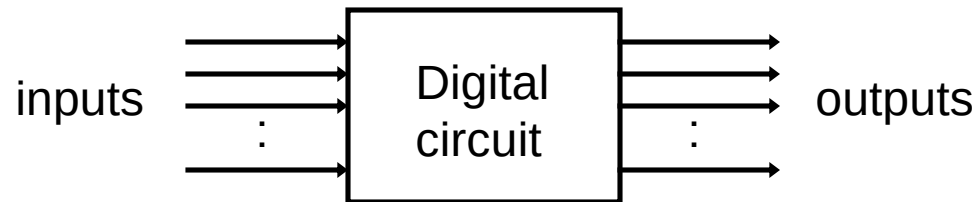
**Boolean algebra** forms the basis of logic circuit design. Consider very simple but common example: if (A is true) and (B is false) then print “the solution is found”. In this case, two Boolean expressions (A is true) and (B is false) are related by a connective ‘and’. How do we define these? This and related things are discussed in this chapter.

In typical circuit design, there are many conditions to be taken care of (for example, when the ‘second counter’ = 60, the ‘minute counter’ is incremented and ‘second counter’ is made 0. Thus it is quite important to understand Boolean algebra. In subsequent chapters, we are going to further study how to minimize the circuit using laws of Boolean algebra (that is very interesting...)

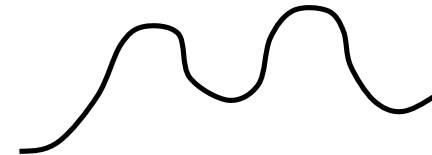
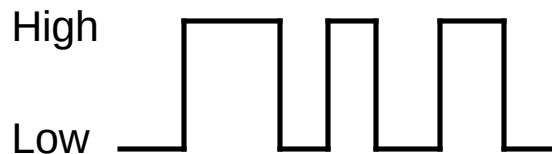


# Digital Circuits

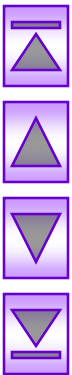
- **Digital circuit** can be represented by a black-box with inputs on one side, and outputs on the other.



The input/output signals are **discrete/digital** in nature, typically with two distinct voltages (a high voltage and a low voltage).

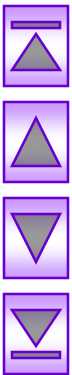


In contrast, **analog circuits** use continuous signals.



# Digital Circuits

- Advantages of Digital Circuits over Analog Circuits:
  - ❖ more reliable (simpler circuits, less noise-prone)
  - ❖ specified accuracy (determinable)
  - ❖ but slower response time (sampling rate)
- Important advantages for two-valued Digital Circuit:
  - ❖ Mathematical Model – Boolean Algebra
  - ❖ Can help *design, analyse, simplify* Digital Circuits.



# Boolean Algebra

What is an **Algebra**? (e.g. algebra of integers)

set of elements (e.g. 0,1,2,...)

set of operations (e.g. +, -, \*,...)

postulates/axioms (e.g.  $0+x=x$ ,...)

- **Boolean Algebra** named after George Boole who used it to study human logical reasoning – calculus of proposition.
- Events : *true* or *false*
- Connectives : a **OR** b; a **AND** b, **NOT** a
- Example: Either “it has rained” **OR** “someone splashed water”, “must be tall” **AND** “good vision”.



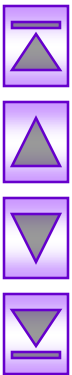
# Boolean Algebra

| a | b | a AND b |
|---|---|---------|
| F | F | F       |
| F | T | F       |
| T | F | F       |
| T | T | T       |

| a | b | a OR b |
|---|---|--------|
| F | F | F      |
| F | T | T      |
| T | F | T      |
| T | T | T      |

| a | NOT a |
|---|-------|
| F | T     |
| T | F     |

Later, Shannon introduced [switching algebra](#) (two-valued Boolean algebra) to represent bi-stable switching circuit.





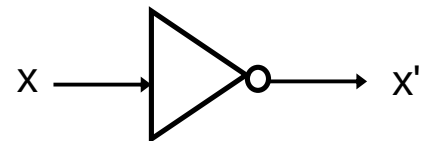
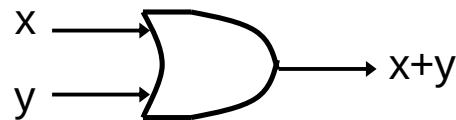
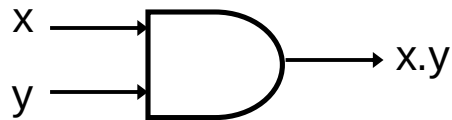
# Two-valued Boolean Algebra

- Set of Elements:  $\{0,1\}$
  - Set of Operations:  $\{., +, \neg\}$
- Sometimes denoted by  $'$ , for example  $a'$

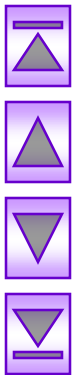
| x | y | $x . y$ |
|---|---|---------|
| 0 | 0 | 0       |
| 0 | 1 | 0       |
| 1 | 0 | 0       |
| 1 | 1 | 1       |

| x | y | $x + y$ |
|---|---|---------|
| 0 | 0 | 0       |
| 0 | 1 | 1       |
| 1 | 0 | 1       |
| 1 | 1 | 1       |

| x | $\neg x$ |
|---|----------|
| 0 | 1        |
| 1 | 0        |



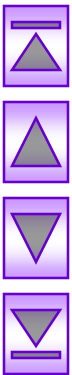
Signals: High = 5V = 1; Low = 0V = 0



# Boolean Algebra Postulates

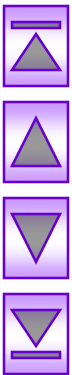
A **Boolean algebra** consists of a set of elements  $B$ , with two binary operations  $\{+\}$  and  $\{.\}$  and a unary operation  $\{'\}$ , such that the following axioms hold:

- The set  $B$  contains at least two distinct elements  $x$  and  $y$ .
- **Closure**: For every  $x, y$  in  $B$ ,
  - ❖  $x + y$  is in  $B$
  - ❖  $x . y$  is in  $B$
- **Commutative laws**: For every  $x, y$  in  $B$ ,
  - ❖  $x + y = y + x$
  - ❖  $x . y = y . x$



# Boolean Algebra Postulates

- **Associative laws:** For every  $x, y, z$  in  $B$ ,
  - ❖  $(x + y) + z = x + (y + z) = x + y + z$
  - ❖  $(x \cdot y) \cdot z = (x \cdot y) \cdot z = x \cdot y \cdot z$
- **Identities** (0 and 1):
  - ❖  $0 + x = x + 0 = x$  for every  $x$  in  $B$
  - ❖  $1 \cdot x = x \cdot 1 = x$  for every  $x$  in  $B$
- **Distributive laws:** For every  $x, y, z$  in  $B$ ,
  - ❖  $x \cdot (y + z) = (x \cdot y) + (x \cdot z)$
  - ❖  $x + (y \cdot z) = (x + y) \cdot (x + z)$



# Boolean Algebra Postulates

- **Complement:** For every  $x$  in  $B$ , there exists an element  $x'$  in  $B$  such that

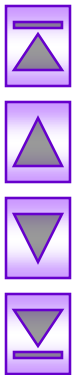
- ❖  $x + x' = 1$

- ❖  $x \cdot x' = 0$

The set  $B = \{0, 1\}$  and the logical operations OR, AND and NOT satisfy all the axioms of a Boolean algebra.

A **Boolean function** maps some inputs over  $\{0,1\}$  into  $\{0,1\}$

A **Boolean expression** is an algebraic statement containing Boolean variables and operators.



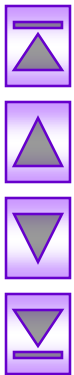
# Precedence of Operators

- To lessen the brackets used in writing boolean expressions, **operator precedence** can be used.
- Precedence (highest to lowest): ' . +
- Examples:

$$a . b + c = (a . b) + c$$

$$b' + c = (b') + c$$

$$a + b' . c = a + ((b') . c)$$

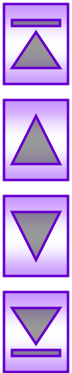


# Precedence of Operators

- Use brackets to overwrite precedence.
- Examples:

$$a . (b + c)$$

$$(a + b)' . c$$



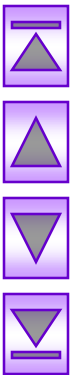
# Truth Table

- Provides a **listing** of every possible combination of inputs and its corresponding outputs.

| INPUTS | OUTPUTS |
|--------|---------|
| ...    | ...     |
| ...    | ...     |

- Example (2 inputs, 2 outputs):

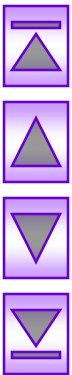
| x | y | $x \cdot y$ | $x + y$ |
|---|---|-------------|---------|
| 0 | 0 | 0           | 0       |
| 0 | 1 | 0           | 1       |
| 1 | 0 | 0           | 1       |
| 1 | 1 | 1           | 1       |



# Truth Table

- Example (3 inputs, 2 outputs):

| x | y | z | $y + z$ | $x.(y + z)$ |
|---|---|---|---------|-------------|
| 0 | 0 | 0 | 0       | 0           |
| 0 | 0 | 1 | 1       | 0           |
| 0 | 1 | 0 | 1       | 0           |
| 0 | 1 | 1 | 1       | 0           |
| 1 | 0 | 0 | 0       | 0           |
| 1 | 0 | 1 | 1       | 1           |
| 1 | 1 | 0 | 1       | 1           |
| 1 | 1 | 1 | 1       | 1           |





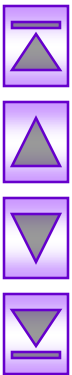
# Proof using Truth Table

- Can use truth table to prove by perfect induction.
  - Prove that:  $x \cdot (y + z) = (x \cdot y) + (x \cdot z)$
- (i) Construct truth table for LHS & RHS of above equality.

| x | y | z | $y + z$ | $x \cdot (y + z)$ | $x \cdot y$ | $x \cdot z$ | $(x \cdot y) + (x \cdot z)$ |
|---|---|---|---------|-------------------|-------------|-------------|-----------------------------|
| 0 | 0 | 0 | 0       | 0                 | 0           | 0           | 0                           |
| 0 | 0 | 1 | 1       | 0                 | 0           | 0           | 0                           |
| 0 | 1 | 0 | 1       | 0                 | 0           | 0           | 0                           |
| 0 | 1 | 1 | 1       | 0                 | 0           | 0           | 0                           |
| 1 | 0 | 0 | 0       | 0                 | 0           | 0           | 0                           |
| 1 | 0 | 1 | 1       | 1                 | 0           | 1           | 1                           |
| 1 | 1 | 0 | 1       | 1                 | 1           | 0           | 1                           |
| 1 | 1 | 1 | 1       | 1                 | 1           | 1           | 1                           |

(ii) Check that LHS = RHS

Postulate is SATISFIED because output column 2 & 5 (for LHS & RHS expressions) are equal for all cases.

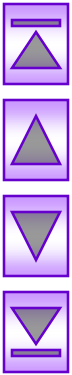


# Solving it yourself (Exercise 3.1)

Q. Use truth table to prove the following.

(a)  $A'.B' + A.B' + A.B = A + B'$

(b)  $A.B.C + A'.B.C + A'.B.C' = B.C + A'.B$



# Duality

- **Duality Principle** – every valid Boolean expression (equality) remains valid if the operators and identity elements are interchanged, as follows:

$$+ \leftrightarrow \cdot$$

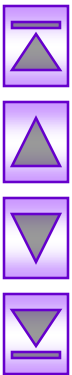
$$1 \leftrightarrow 0$$

- Example: Given the expression

$$a + (b \cdot c) = (a + b) \cdot (a + c)$$

then its **dual expression** is

$$a \cdot (b + c) = (a \cdot b) + (a \cdot c)$$



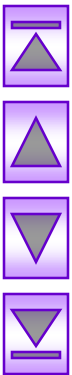
# Duality

- Duality gives free theorems – “two for the price of one”. You prove one theorem and the other comes for free!
- If  $(x+y+z)' = x'.y'.z'$  is valid, then its dual is also valid:

$$(x.y.z)' = x'+y'+z'$$

- If  $x + 1 = 1$  is valid, then its dual is also valid:

$$x . 0 = 0$$



# Basic Theorems of Boolean Algebra

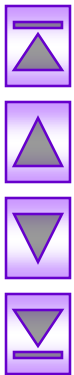
- Apart from the axioms/postulates, there are other useful theorems.

## 1. Idempotency.

$$(a) \ x + x = x \qquad (b) \ x \cdot x = x$$

Proof of (a):

$$\begin{aligned} x + x &= (x + x) \cdot 1 && \text{(identity)} \\ &= (x + x) \cdot (x + x') && \text{(complementarity)} \\ &= x + x \cdot x' && \text{(distributivity)} \\ &= x + 0 && \text{(complementarity)} \\ &= x && \text{(identity)} \end{aligned}$$



# Basic Theorems of Boolean Algebra

2. **Null elements** for + and . operators.

$$(a) \ x + 1 = 1$$

$$(b) \ x \cdot 0 = 0$$

3. **Involution.**  $(x')' = x$

4. **Absorption.**

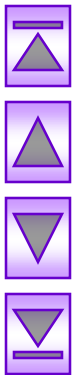
$$(a) \ x + x \cdot y = x$$

$$(b) \ x \cdot (x + y) = x$$

5. **Absorption** (variant).

$$(a) \ x + x' \cdot y = x + y$$

$$(b) \ x \cdot (x' + y) = x \cdot y$$



# Basic Theorems of Boolean Algebra

## 6. DeMorgan.

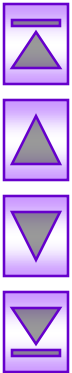
$$(a) (x + y)' = x'.y'$$

$$(b) (x.y)' = x' + y'$$

## 7. Consensus.

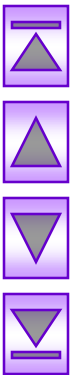
$$(a) x.y + x'.z + y.z = x.y + x'.z$$

$$(b) (x+y).(x'+z).(y+z) = (x+y).(x'+z)$$



# Basic Theorems of Boolean Algebra

- Theorems can be proved using the truth table method. (Exercise: Prove De-Morgan's theorem using the truth table.)
- They can also be proved by **algebraic manipulation** using axioms/postulates or other basic theorems.





# Basic Theorems of Boolean Algebra

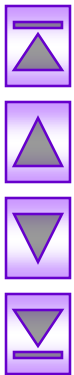
- Theorem 4a (absorption) can be proved by:

$$\begin{aligned}x + x.y &= x.1 + x.y && \text{(identity)} \\&= x.(1 + y) && \text{(distributivity)} \\&= x.(y + 1) && \text{(commutativity)} \\&= x.1 && \text{(Theorem 2a)} \\&= x && \text{(identity)}\end{aligned}$$

- By duality, theorem 4b:

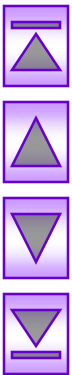
$$x.(x+y) = x$$

- Try prove this by algebraic manipulation.



# Boolean Functions

- **Boolean function** is an expression formed with binary variables, the two binary operators, OR and AND, and the unary operator, NOT, parenthesis and the equal sign.
- Its result is also a binary value.
- We usually use  $\cdot$  for AND,  $+$  for OR, and  $'$  or  $\neg$  for NOT. Sometimes, we may omit the  $\cdot$  if there is no ambiguity.



# Boolean Functions

- Examples:

$$F1 = xyz'$$

$$F2 = x + y'z$$

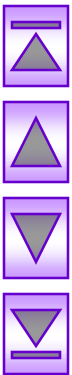
$$F3 = (x'y'z) + (x'yz) + (xy')$$

$$F4 = xy' + x'z$$

| x | y | z | F1 | F2 | F3 | F4 |
|---|---|---|----|----|----|----|
| 0 | 0 | 0 | 0  | 0  | 0  | 0  |
| 0 | 0 | 1 | 0  | 1  | 1  | 1  |
| 0 | 1 | 0 | 0  | 0  | 0  | 0  |
| 0 | 1 | 1 | 0  | 0  | 1  | 1  |
| 1 | 0 | 0 | 0  | 1  | 1  | 1  |
| 1 | 0 | 1 | 0  | 1  | 1  | 1  |
| 1 | 1 | 0 | 1  | 1  | 0  | 0  |
| 1 | 1 | 1 | 0  | 1  | 0  | 0  |

From the truth table,  $F3 = F4$ .

Can you also prove by algebraic manipulation that  $F3 = F4$ ?



# Complement of Functions

- Given a function,  $F$ , the **complement** of this function,  $F'$ , is obtained by interchanging 1 with 0 in the function's output values.

Example:  $F1 = xyz'$

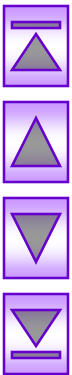
Complement:

$$F1' = (xyz')'$$

$$= x' + y' + (z')' \quad \text{DeMorgan}$$

$$= x' + y' + z \quad \text{Involution}$$

| x | y | z | F1 | F1' |
|---|---|---|----|-----|
| 0 | 0 | 0 | 0  | 1   |
| 0 | 0 | 1 | 0  | 1   |
| 0 | 1 | 0 | 0  | 1   |
| 0 | 1 | 1 | 0  | 1   |
| 1 | 0 | 0 | 0  | 1   |
| 1 | 0 | 1 | 0  | 1   |
| 1 | 1 | 0 | 1  | 0   |
| 1 | 1 | 1 | 0  | 1   |

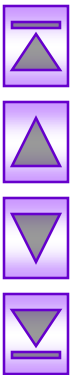


# Complement of Functions

- More general DeMorgan's theorems useful for obtaining complement functions:

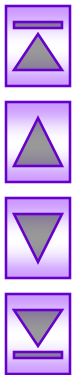
$$(A + B + C + \dots + Z)' = A' \cdot B' \cdot C' \dots \cdot Z'$$

$$(A \cdot B \cdot C \dots \cdot Z)' = A' + B' + C' + \dots + Z'$$



# Standard Forms

- Certain types of Boolean expressions lead to gating networks which are desirable from implementation viewpoint.
- Two Standard Forms:  
*Sum-of-Products* and *Product-of-Sums*
- **Literals**: a variable on its own or in its complemented form. Examples:  $x$ ,  $x'$ ,  $y$ ,  $y'$
- **Product Term**: a single literal or a logical product (AND) of several literals.  
Examples:  $x$ ,  $xyz'$ ,  $A'B$ ,  $AB$



# Standard Forms

- **Sum Term**: a single literal or a logical sum (OR) of several literals.

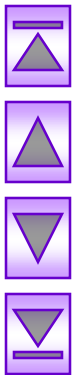
Examples:  $x$ ,  $x+y+z'$ ,  $A'+B$ ,  $A+B$

- **Sum-of-Products (SOP) Expression**: a product term or a logical sum (OR) of several product terms.

Examples:  $x$ ,  $x+yz'$ ,  $xy'+x'yz$ ,  $AB+A'B'$

- **Product-of-Sums (POS) Expression**: a sum term or a logical product (AND) of several sum terms.

Examples:  $x$ ,  $x(y+z')$ ,  $(x+y')(x'+y+z)$ ,  $(A+B)(A'+B')$



# Standard Forms

- Every boolean expression can either be expressed as sum-of-products or product-of-sums expression.

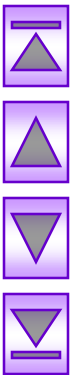
Examples:

SOP:  $x'y + xy' + xyz$

POS:  $(x + y')(x' + y)(x' + z')$

both:  $x' + y + z$  or  $xyz'$

neither:  $x(w' + yz)$  or  $z' + wx'y + v(xz + w')$



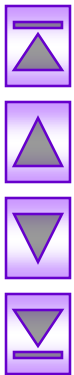


# Minterm & Maxterm

- Consider two binary variables  $x, y$ .
- Each variable may appear as itself or in complemented form as literals (i.e.  $x, x'$  &  $y, y'$  )
- For two variables, there are four possible combinations with the AND operator, namely:

$$x'y', x'y, xy', xy$$

- These product terms are called the *minterms*.
- A **minterm** of  $n$  variables is the product of  $n$  literals from the different variables.

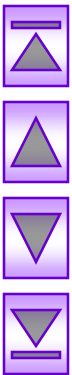


# Minterm & Maxterm

- In general,  $n$  variables can give  $2^n$  minterms.
- In a similar fashion, a **maxterm** of  $n$  variables is the sum of  $n$  literals from the different variables.

Examples:  $x'+y'$ ,  $x'+y$ ,  $x+y'$ ,  $x+y$

- In general,  $n$  variables can give  $2^n$  maxterms.



# Minterm & Maxterm

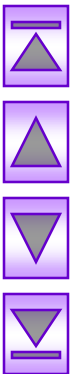
- The minterms and maxterms of 2 variables are denoted by m0 to m3 and M0 to M3 respectively:

| Minterms |   |        |          | Maxterms |          |
|----------|---|--------|----------|----------|----------|
| x        | y | term   | notation | term     | notation |
| 0        | 0 | $x'y'$ | m0       | $x+y$    | M0       |
| 0        | 1 | $x'y$  | m1       | $x+y'$   | M1       |
| 1        | 0 | $xy'$  | m2       | $x'+y$   | M2       |
| 1        | 1 | $xy$   | m3       | $x'+y'$  | M3       |

Each minterm is the **complement** of the corresponding maxterm:

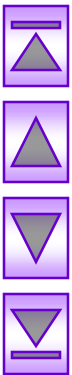
Example:  $m2 = xy'$

$$m2' = (xy')' = x' + (y')' = x' + y = M2$$



# Canonical Form: Sum of Minterms

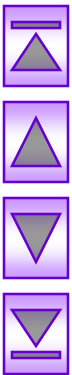
- What is a **canonical/normal form**?
  - ❖ A unique form for representing something.
- Minterms are product terms.
  - ❖ Can express Boolean functions using Sum-of-Minterms form.



# Canonical Form: Sum of Minterms

a) Obtain the truth table. Example:

| x | y | z | F1 | F2 | F3 |
|---|---|---|----|----|----|
| 0 | 0 | 0 | 0  | 0  | 0  |
| 0 | 0 | 1 | 0  | 1  | 1  |
| 0 | 1 | 0 | 0  | 0  | 0  |
| 0 | 1 | 1 | 0  | 0  | 1  |
| 1 | 0 | 0 | 0  | 1  | 1  |
| 1 | 0 | 1 | 0  | 1  | 1  |
| 1 | 1 | 0 | 1  | 1  | 0  |
| 1 | 1 | 1 | 0  | 1  | 0  |



# Canonical Form: Sum of Minterms

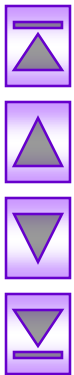
b) Obtain Sum-of-Minterms by gathering/summing the minterms of the function (where result is a 1)

$$F1 = xyz' = \Sigma(m6)$$

$$F2 = x'y'z + xy'z' + xy'z + xyz' + xyz = \Sigma(m1, m4, m5, m6, m7)$$

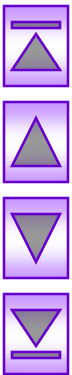
$$\begin{aligned} F3 &= x'y'z + x'yz + xy'z' \\ &\quad + xy'z \\ &= \Sigma(m1, m3, m4, m5) \end{aligned}$$

| x | y | z | F1 | F2 | F3 |
|---|---|---|----|----|----|
| 0 | 0 | 0 | 0  | 0  | 0  |
| 0 | 0 | 1 | 0  | 1  | 1  |
| 0 | 1 | 0 | 0  | 0  | 0  |
| 0 | 1 | 1 | 0  | 0  | 1  |
| 1 | 0 | 0 | 0  | 1  | 1  |
| 1 | 0 | 1 | 0  | 1  | 1  |
| 1 | 1 | 0 | 1  | 1  | 0  |
| 1 | 1 | 1 | 0  | 1  | 0  |



# Canonical Form: Product of Maxterms

- Maxterms are sum terms.
- For Boolean functions, the maxterms of a function are the terms for which the result is 0.
- Boolean functions can be expressed as Products-of-Maxterms.



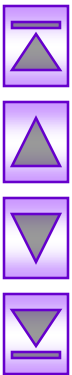
# Canonical Form: Product of Maxterms

E.g.:  $F2 = \Pi(M0, M2, M3) = (x+y+z)(x+y'+z)(x+y'+z')$

$$F3 = \Pi(M0, M2, M6, M7)$$

$$= (x+y+z)(x+y'+z)(x'+y'+z)(x'+y'+z')$$

| x | y | z | F1 | F2 | F3 |
|---|---|---|----|----|----|
| 0 | 0 | 0 | 0  | 0  | 0  |
| 0 | 0 | 1 | 0  | 1  | 1  |
| 0 | 1 | 0 | 0  | 0  | 0  |
| 0 | 1 | 1 | 0  | 0  | 1  |
| 1 | 0 | 0 | 0  | 1  | 1  |
| 1 | 0 | 1 | 0  | 1  | 1  |
| 1 | 1 | 0 | 1  | 1  | 0  |
| 1 | 1 | 1 | 0  | 1  | 0  |





# Canonical Form: Product of Maxterms

- Why is this so? Take F2 as an example.

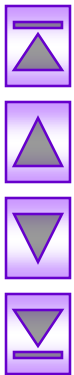
$$F2 = \Sigma(m1, m4, m5, m6, m7)$$

- The complement function of F2 is:

$$\begin{aligned} F2' &= \Sigma(m0, m2, m3) \\ &= m0 + m2 + m3 \end{aligned}$$

| x | y | z | F2 | F2' |
|---|---|---|----|-----|
| 0 | 0 | 0 | 0  | 1   |
| 0 | 0 | 1 | 1  | 0   |
| 0 | 1 | 0 | 0  | 1   |
| 0 | 1 | 1 | 0  | 1   |
| 1 | 0 | 0 | 1  | 0   |
| 1 | 0 | 1 | 1  | 0   |
| 1 | 1 | 0 | 1  | 0   |
| 1 | 1 | 1 | 1  | 0   |

(Complement functions' minterms are the opposite of their original functions, i.e. when original function = 0)



# Canonical Form: Product of Maxterms

From previous slide,  $F2' = m0 + m2 + m3$

Therefore:

$$F2 = (m0 + m2 + m3)'$$

$$= m0' \cdot m2' \cdot m3'$$

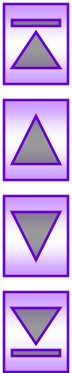
DeMorgan

$$= M0 \cdot M2 \cdot M3$$

$$mx' = Mx$$

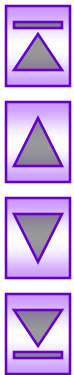
$$= \Pi(M0, M2, M3)$$

- Every Boolean function can be expressed as either Sum-of-Minterms or Product-of-Maxterms.



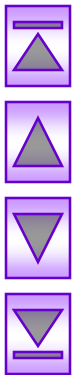
# Solving it yourself (Exercise 3.2)

1. A Boolean function of 5 variables can have up to \_\_\_\_ minterms.
2. Given a Boolean function  $F(A,B,C) = \sum m(0, 5, 7)$ .  
Which of the following is correct?
  - a)  $F'(A,B,C) = \sum m(0,5,7)$
  - b)  $F(A,B,C) = \prod M(1,2,3,4,6)$
  - c)  $F(A,B,C) = \prod M(0,5,7)$
  - d)  $F'(A,B,C) = \sum m(1,2,3)$
  - e) None above
3. Given a Boolean function  $F(x,y,z) = y' \cdot (x + z') + x' \cdot z$ .  
Which of the following is correct?
  - a)  $F(x,y,z) = \sum m(0,1)$
  - b)  $F(x,y,z) = \sum m(0,1,4,5)$
  - c)  $F(x,y,z) = \sum m(0,1,2,3,4)$
  - d)  $F(x,y,z) = \sum m(0,1,3,4,5)$
  - e)  $F(x,y,z) = \sum m(1,2,3,4,5)$



# Solving it yourself (Exercise 3.2)

4. Given a Boolean function with 6 variables  $a, b, c, d, e, f$ .  
What is maxterm  $M_{60}$ ?
- a)  $a'.b'.c'.d'.e.f$     b)  $a'+b'+c'+d'$     c)  $a'+b'+c'+d'+e+f$   
d)  $a.b.c.d.e'.f'$     e)  $a+b+c+d+e'+f'$
5. Given a Boolean function  $F(a,b,c) = \sum m(0, 2, 5, 6, 7)$ .  
If  $a=0, b=c=1$ , what is the value of  $F$ ?
- a) 0    b) 1    c)  $b.c$     d)  $a'$     e) Unknown
6. Which of the following is NOT a minterm of the Boolean function:  $F(w,x,y,z) = w.x.z' + x.y'.z + x.z$
- a)  $w.x.y.z'$     b)  $w'.x.y'.z$     c)  $w.x.y.z$   
d)  $w.x.y'.z'$     e)  $w.x'.y.z$

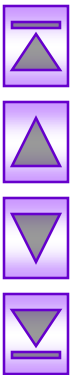


# Solving it yourself (Exercise 3.2)

7. Identify the following function  $F(x,y,z)$ .

- a)  $x.y'.z' + x'.y.z + x.z'$
- b)  $x'.y.z + x.y' + x'.y'.z$
- c)  $x.y'.z + x'.y'.z + x.y$
- d)  $x.y.z + x'.y'.z' + x.y'$
- e) None above

| $x$ | $y$ | $z$ | $F$ |
|-----|-----|-----|-----|
| 0   | 0   | 0   | 0   |
| 0   | 0   | 1   | 1   |
| 0   | 1   | 0   | 0   |
| 0   | 1   | 1   | 1   |
| 1   | 0   | 0   | 1   |
| 1   | 0   | 1   | 1   |
| 1   | 1   | 0   | 0   |
| 1   | 1   | 1   | 0   |



# Conversion of Canonical Forms

## ■ Sum-of-Minterms $\Rightarrow$ Product-of-Maxterms

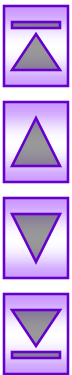
- ❖ Rewrite minterm shorthand using maxterm shorthand.
- ❖ Replace minterm indices with indices not already used.

$$\text{Eg: } F1(A,B,C) = \sum m(3,4,5,6,7) = \prod M(0,1,2)$$

## ■ Product-of-Maxterms $\Rightarrow$ Sum-of-Minterms

- ❖ Rewrite maxterm shorthand using minterm shorthand.
- ❖ Replace maxterm indices with indices not already used.

$$\text{Eg: } F2(A,B,C) = \prod M(0,3,5,6) = \sum m(1,2,4,7)$$



# Conversion of Canonical Forms

## ■ Sum-of-Minterms of $F \Rightarrow$ Sum-of-Minterms of $F'$

- ❖ In minterm shorthand form, list the indices not already used in  $F$ .

$$\text{Eg: } F1(A,B,C) = \sum m(3,4,5,6,7)$$

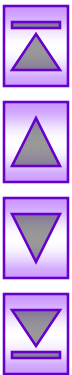
$$F1'(A,B,C) = \sum m(0,1,2)$$

## ■ Product-of-Maxterms of $F \Rightarrow$ Prod-of-Maxterms of $F'$

- ❖ In maxterm shorthand form, list the indices not already used in  $F$ .

$$\text{Eg: } F1(A,B,C) = \prod M(0,1,2)$$

$$F1'(A,B,C) = \prod M(3,4,5,6,7)$$



# Conversion of Canonical Forms

## ■ Sum-of-Minterms of $F \Rightarrow$ Product-of-Maxterms of $F'$

- ❖ Rewrite in maxterm shorthand form, using the same indices as in  $F$ .

$$\text{Eg: } F1(A,B,C) = \sum m(3,4,5,6,7)$$

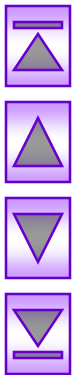
$$F1'(A,B,C) = \prod M(3,4,5,6,7)$$

## ■ Product-of-Maxterms of $F \Rightarrow$ Sum-of-Minterms of $F'$

- ❖ Rewrite in minterm shorthand form, using the same indices as in  $F$ .

$$\text{Eg: } F1(A,B,C) = \prod M(0,1,2)$$

$$F1'(A,B,C) = \sum m(0,1,2)$$



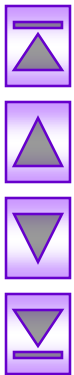


# Solving it yourself (Exercise 3.3)

Q. Consider the function

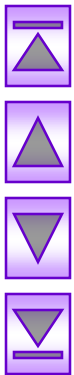
$$f(A,B,Y,Z) = \Sigma m(0, 2, 5, 6, 8, 11, 15).$$

- a) Write this as a Boolean expression in canonical SOP form.
- b) Rewrite the expression in canonical POS form.
- c) Write the complement of  $f$  in “little m” notation and in canonical SOP form.
- d) Write the complement of  $f$  in “big M” notation and in canonical POS form.



# Binary Functions

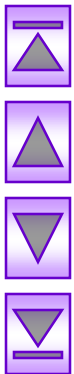
- Given  $n$  variables, there are  $2^n$  possible minterms.
- As each function can be expressed as sum-of-minterms, there could be  $2^{2^n}$  different functions.
- In the case of two variables, there are  $2^2=4$  possible minterms; and  $2^4=16$  different possible binary functions.
- The 16 possible binary functions are shown in the next slide.



# Binary Functions

| x      | y | F <sub>0</sub> | F <sub>1</sub> | F <sub>2</sub> | F <sub>3</sub> | F <sub>4</sub> | F <sub>5</sub> | F <sub>6</sub> | F <sub>7</sub> |
|--------|---|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|
| 0      | 0 | 0              | 0              | 0              | 0              | 0              | 0              | 0              | 0              |
| 0      | 1 | 0              | 0              | 0              | 0              | 1              | 1              | 1              | 1              |
| 1      | 0 | 0              | 0              | 1              | 1              | 0              | 0              | 1              | 1              |
| 1      | 1 | 0              | 1              | 0              | 1              | 0              | 1              | 0              | 1              |
| Symbol |   | .              | /              |                | /              |                |                | ⊕              | +              |
| Name   |   | AND            |                |                |                |                | XOR            |                | OR             |

| x      | y | $F_8$        | $F_9$   | $F_{10}$ | $F_{11}$  | $F_{12}$ | $F_{13}$  | $F_{14}$   | $F_{15}$ |
|--------|---|--------------|---------|----------|-----------|----------|-----------|------------|----------|
| 0      | 0 | 1            | 1       | 1        | 1         | 1        | 1         | 1          | 1        |
| 0      | 1 | 0            | 0       | 0        | 0         | 1        | 1         | 1          | 1        |
| 1      | 0 | 0            | 0       | 1        | 1         | 0        | 0         | 1          | 1        |
| 1      | 1 | 0            | 1       | 0        | 1         | 0        | 1         | 0          | 1        |
| Symbol |   | $\downarrow$ | $\odot$ | '        | $\subset$ | '        | $\supset$ | $\uparrow$ |          |
| Name   |   | NOR          | XNOR    | NAND     |           |          |           |            |          |



# End of segment

